

Burp Suite Lab Report

SQL Injection & Authentication Labs

Completed 8 hands-on labs on PortSwigger's Web Security Academy using Burp Suite: 5 SQL injection labs and 3 Authentication labs. All labs below were solved.

1. SQL Injection Labs (5)

APPRENTICE

SQL injection vulnerability in WHERE clause allowing retrieval of hidden data

✓ Solved

Injected SQL into a query parameter used inside a WHERE clause to alter the logic of the query (e.g. appending OR 1=1) so that the application returned hidden/unauthorized rows of data that shouldn't normally be visible.

Steps to solve:

1. Browsed the target site and identified a parameter that filters results (e.g. a category filter in a product listing URL).
2. Sent the request to Burp Repeater to test it directly.
3. Modified the parameter value to break out of the query logic, appending a condition that is always true (e.g. combining it with OR 1=1 using a comment to remove the rest of the original query).
4. Sent the modified request and confirmed the response now returned additional/hidden rows that weren't shown in the normal filtered view.
5. Verified the fix by loading the modified request in the browser and confirming the hidden data was visible on the page.

APPRENTICE

SQL injection vulnerability allowing login bypass

✓ Solved

Bypassed a login form by injecting SQL into the username/password fields so the WHERE clause always evaluated true, authenticating without knowing valid credentials (a classic 'OR '1'='1'-- style bypass).

Steps to solve:

1. Captured the login request in Burp Proxy while submitting the login form.
2. Sent the request to Repeater to iterate on the username field.
3. Entered a known/guessed username followed by a comment sequence to strip out the password check from the underlying query.
4. Left the password field as any value, since the injected comment causes it to be ignored by the query.
5. Sent the request and confirmed a successful login response (redirect/session set) instead of an authentication failure.

PRACTITIONER

SQL injection UNION attack, determining the number of columns returned by the query

✓ Solved

Used ORDER BY and UNION SELECT NULL,NULL... with an increasing number of NULLs to determine how many columns the original query returns, which is required before extending it with a UNION attack.

Steps to solve:

1. Identified an injectable parameter and sent the request to Repeater.
2. Injected an ORDER BY clause with an increasing column number until the response returned an error, indicating the column count had been exceeded.
3. Alternatively, injected a UNION SELECT NULL statement, adding one more NULL at a time.
4. Noted the point where the error disappeared (ORDER BY) or the query succeeded (UNION SELECT) — that number of columns matches the original query.

PRACTITIONER

SQL injection UNION attack, finding a column containing text

✓ Solved

Once column count was known, substituted a string value (e.g. 'a') into each column position via UNION SELECT to identify which column(s) accept text data, needed to display extracted data back on the page.

Steps to solve:

1. Using the column count found previously, built a UNION SELECT with that many values.
2. Replaced each position one at a time with a short string value (e.g. 'a') while keeping the others as NULL.
3. Sent each variation and checked which one returned a successful response instead of a type-mismatch error.
4. Confirmed the column(s) that accepted text — these are the positions usable to display extracted string data later.

PRACTITIONER

SQL injection UNION attack, retrieving data from other tables

✓ Solved

Used the confirmed text-compatible column with a UNION SELECT against a different table (e.g. querying usernames/passwords from a users table) to exfiltrate data the application was never designed to display.

Steps to solve:

1. Confirmed the number of columns and which column accepted text, from the previous two labs.
2. Replaced the text-compatible column's value with a subquery/column name from the target table (e.g. selecting username and password from a users table).
3. Combined multiple values into one column where needed, using a separator (e.g. concatenating username and password together) since only one text column was available.
4. Sent the request and read the extracted credentials back from the rendered page.

2. Authentication Labs (3)

APPRENTICE

Username enumeration via different responses

✓ Solved

Compared the application's responses to valid vs. invalid usernames during login (differences in error messages, status codes, or response timing) to determine which usernames exist on the system.

Steps to solve:

1. Submitted the login form with an obviously invalid username/password and captured the response in Burp.
2. Sent the request to Intruder and marked the username field as the payload position.
3. Loaded a wordlist of candidate usernames and ran a Sniper attack.
4. Compared response length/content/status across all results — the entries that differed (e.g. a different error message for 'invalid password' vs 'invalid username') revealed which usernames actually exist.
5. Took a confirmed valid username and repeated a similar process against the password field to complete the login.

APPRENTICE

2FA simple bypass

✓ Solved

Identified a broken two-factor authentication flow where the second verification step could be skipped entirely — e.g. by navigating directly to a post-login page instead of completing the 2FA code step, exposing a logic flaw rather than cracking the actual code.

Steps to solve:

1. Logged in with valid credentials and observed the flow being redirected to a 2FA verification page.
2. Inspected the URL/flow in Burp Proxy to see what page the app intended to load only after 2FA success.
3. Instead of submitting a 2FA code, navigated directly to that post-login/account page.
4. Confirmed the application granted access without ever validating the 2FA code, revealing that the second factor was only enforced client-side or via a page redirect, not on the server session state.

APPRENTICE

Password reset broken logic

✓ Solved

Exploited flawed logic in the password-reset flow (such as a parameter identifying which account to reset that could be tampered with) to reset another user's password without proving ownership of their account.

Steps to solve:

1. Triggered the password-reset flow for an account under my control and captured the reset request in Burp Proxy.
2. Inspected the request/response for a parameter that identifies which user the reset applies to (e.g. a hidden username/user-id field).
3. Sent the request to Repeater and modified that parameter to point to a different, target username.
4. Submitted the modified request and confirmed the application reset the target account's password instead of rejecting the mismatched identity.

3. Summary

These 8 labs cover two of the most common real-world web vulnerability classes: SQL injection (from basic filter bypass through to full UNION-based data extraction) and broken authentication logic (enumeration, weak 2FA enforcement, and flawed password-reset flows). All labs were completed and marked Solved on the Web Security Academy platform.